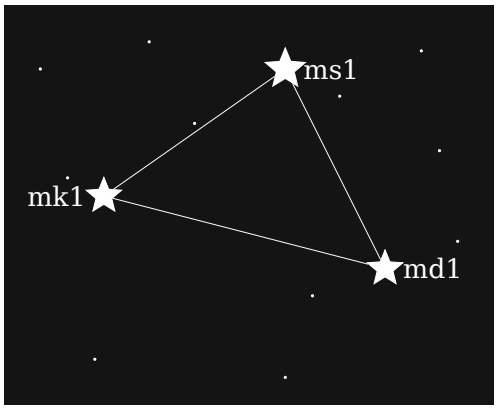
m-format Constellation Quick Start

Engrave your first 3-card backup in 90 minutes

bg002h

May 8, 2026



Contents

Read this first — UNTESTED ALPHA SOFTWARE

This software has not yet been independently tested, audited, or proven in production. The m-format constellation is alpha-quality work-in-progress: codecs, CLIs, BCH error-correcting math, BIP-388 wallet-policy emission, and the cross-card binding invariants have all been authored and reviewed only by the original developer.

Do not use this software to back up significant sums of money at this time. Doing so is tantamount to asking to be rekt.

Acceptable uses today: disposable amounts only (a few sats), on mainnet or testnet; evaluation; learning; code review; reproducing the published worked-example transcripts; integration smoke-testing.

Unacceptable uses today: production wallets covering meaningful balances; inheritance plans; any wallet you would be unhappy to lose.

The format itself is intended to mature through external review and independent implementation. Until that review happens, assume bugs. When the project reaches a stable release with documented external review, this page will be replaced with a stability-claim page.

About this Quick Start

This is a hands-on onboarding to the **m-format constellation** — a family of checksum-protected backup cards designed for steel engraving. By the end you will have produced a working multi-card Bitcoin backup, and you will understand what each card does.

Audience

You have heard of Bitcoin self-custody — the idea that *you* hold the keys to your coins, not an exchange — and you want to engrave your first multi-card backup. No prior Bitcoin background is required. Everything technical is introduced as you need it.

Prerequisites

- A Linux or macOS terminal. (Windows works via WSL but is not covered here.)
- Basic comfort with shell commands: copying a command, running it, reading the output.
- Roughly an hour of uninterrupted time.

You do not need: existing Bitcoin software, a hardware wallet, or prior cryptography knowledge.

What you'll have at the end

Three things, depending on how far you read:

- **Part II (single-sig).** A complete steel-engraved bundle for a single-signature wallet — the simplest setup that the m-format star supports.
- **Part III (multisig).** A 2-of-3 multisig bundle: three independent cards that together describe a wallet requiring two signatures of three to spend.
- **Part IV (watch-only).** Configurations that import either of the above into popular wallet software (Sparrow, Bitcoin Core) for monitoring without exposing the secret.

Reading order

Top to bottom, roughly 90 minutes. Each chapter forward-points to the next; later chapters assume the earlier ones. Skim if you are already comfortable with a section's topic, but do not skip out of order — the worked examples build on each other.

What this guide is not

This is not the full reference. For exhaustive coverage of every CLI flag, every supported descriptor type, the BIP-85 child-secret flow, taproot multisig, or recovery edge cases, see the **reference manual** (`docs/manual/`) — a 129-page sibling document that this Quick Start cross-references.

Onward to the foundations.

What you're building

The problem in one paragraph

Self-custodied Bitcoin starts with a *seed phrase* — a sequence of 12 to 24 English words from which every key in your wallet is derived. People engrave seed phrases on steel so a house fire or flood does not destroy the backup. But a plain seed phrase on steel has no built-in error detection: a single mis-stamped letter can silently produce a valid-looking but *different* phrase, and you might not notice until you try to recover. Worse, a seed phrase alone is not enough to recover a multisig wallet — you also need to remember the wallet's spending rule and your cosigners' public keys.

The m-format answer in one paragraph

The m-format constellation splits the backup into **three independently checksummed cards**, each engraved on its own steel plate:

- a **secret card** (`ms1`) carrying the random bits behind your seed phrase,
- a **key card** (`mk1`) carrying a public key (an *xpub*),
- a **descriptor card** (`md1`) carrying the wallet's spending rule.

Each card has its own error-correcting checksum, so a stamping mistake is detectable and locatable. A fourth piece — the `mnemonic-toolkit` command-line tool — synthesizes all three from your inputs and verifies they belong together.

The four pieces

The toolkit is what you run on your computer. It does not engrave itself onto steel; the three cards do. You produce all three from the toolkit, engrave them, and store them — together for a single- signature wallet, or distributed across cosigners for multisig.

What this guide covers

- **Part II (single-sig).** Generate a seed, produce all three cards, verify them, engrave them, then practice recovery.
- **Part III (multisig).** Build a 2-of-3 multisig: three cosigners each contribute their own key, and the toolkit binds them together into a coherent backup.

- **Part IV (watch-only).** Import either of the above into wallet software that watches the addresses without holding the secret.
- **Part V (next steps).** Pointers into the reference manual for the topics this Quick Start does not cover: taproot multisig, BIP-85 children, advanced recovery, and more.

Onward: a 30-second Bitcoin orientation that fills in the few terms used above.

Bitcoin in 30 seconds

Three concepts power every modern Bitcoin wallet: a *seed phrase*, an *extended public key*, and a *wallet descriptor*. This chapter introduces each in three or four sentences. The m-format constellation encodes all three.

Seed phrase

A seed phrase is a sequence of 12 to 24 English words that encodes a random secret. The standard for this encoding is **BIP-39**: roughly, you pick 128 or 256 random bits, compute a short checksum of them, append it, and slice the combined bitstream into 11-bit chunks that index into a fixed 2048-word list. Almost every Bitcoin wallet today understands BIP-39 phrases. The seed phrase is the only thing you actually need to keep secret — every key in your wallet derives from it.

Extended public key (xpub)

From the seed, your wallet computes a *master key*, then derives a tree of child keys via **BIP-32**. Each child has both a private form (used to sign transactions) and a public form (used to receive to addresses). An **xpub** is a node in that tree exported in its public-only form: you can hand an xpub to wallet software, and it can derive *every receive address* the wallet will ever use, without ever seeing the secret. xpubs are how watch-only wallets work; they are also what cosigners share with each other in multisig setups.

Wallet descriptor

A descriptor is a one-line text recipe describing a wallet's spending rule completely. Example shape (the parts will be explained later):

```
wpkh([fingerprint/84h/0h/0h]xpub6Cat.../<0;1>/*)
```

That string says: "this is a single-key, native-segwit wallet (`wpkh`); the key is the xpub shown; both receive addresses (chain 0) and change addresses (chain 1) are valid." Multisig descriptors look the same but with multiple keys and a threshold, e.g. `wsh(sortedmulti(2, xpub_A, xpub_B, xpub_C))`. **BIP-388** extends descriptors into a *wallet policy* format that hardware wallets and the m-format `md1` card use: it

splits the *template* (the recipe shape) from the *bound keys* (which xpub goes into which slot).

BIP, what's a BIP?

> A **BIP** — *Bitcoin Improvement Proposal* — is a numbered specification document. Bitcoin's standards process publishes each protocol or wallet convention as a BIP at https://github.com/bitcoin/bips. When this guide says "BIP-39" or "BIP-388" it means a specific document there. The m-format star uses several BIPs as building blocks: BIP-39 for entropy, BIP-32 for key derivation, BIP-388 for wallet policies, and BIP-93 ("codex32") for the error-correcting alphabet engraved on each card.

Onward: how those three concepts map onto the three m-format cards.

The three cards: ms1, mk1, md1

The m-format constellation maps the three Bitcoin concepts from the previous chapter onto three cards, one per concept.

One card per concept

- **`ms1` — the secret card.** Carries the random bits behind your seed phrase (the BIP-39 *entropy*, before the words). With the `ms1` card alone you can regenerate the same seed phrase your wallet was created from.
- **`mk1` — the key card.** Carries one xpub plus its *origin* — a 4-byte master-fingerprint identifier and the BIP-32 path used to derive the xpub. (The fingerprint and path together pin the xpub to a specific position in the master tree.) With one or more `mk1` cards you can rebuild the public side of a wallet.
- **`md1` — the descriptor card.** Carries the wallet's spending rule as a BIP-388 wallet policy — a *template* (e.g. `wsh(sortedmulti(2,@0/**,@1/**,@2/**))`) plus *one* bound xpub for the slot this card represents. With the `md1` card you know what kind of wallet to recover into.

What each card answers

Card	Carries	What it answers
`ms1`	BIP-39 entropy	"What were the seed words?"
`mk1`	xpub + origin (master fingerprint + BIP path)	"What public key did the wallet use, and at which derivation?"
`md1`	wallet policy (template + one bound xpub)	"What was the wallet's *spending rule* — single-sig? 2-of-3 multisig?"

For a **single-signature** wallet you produce one of each: one `ms1`, one `mk1`, one `md1`. For a **2-of-3 multisig** the secret holders each get their own `ms1` card on their own machine; each cosigner contributes one `mk1` (their xpub); the coordinator produces a single `md1` describing the 2-of-3 policy.

Why three cards instead of one

A naked seed phrase recovers a single-sig wallet *if and only if* the recovery software can guess the spending rule (BIP-84 native segwit, in practice). For a multisig wallet — say a 2-of-3 with three cosigners' xpubs and a custom path — the seed alone is insufficient. You need the **secret** (`ms1`) to sign, the **public keys** (one `mk1` per cosigner) to construct the multisig script, and the **policy** (`md1`) to know *what kind* of multisig.

Splitting the backup across three independently-checksummed steel cards gives two distinct kinds of resilience.

Per-card BCH error correction. Each card carries its own codex32-pattern BCH checksum. Per BIP-93 §"Error Correction", the guarantees per card are:

- **detection:** any error affecting up to 8 characters per card, guaranteed (random patterns beyond that miss with probability $< 3 \times 10^{-20}$);
- **correction (substitutions):** up to 4 wrong characters at unknown positions can be reconstructed;
- **correction (erasures):** up to 8 characters at known positions (e.g., illegible smudges) can be reconstructed; or up to 13 in a single consecutive run (a scratch).

An erasure means *the position is preserved, the value is unknown*. **Deletions and insertions** (length-changing damage) are outside this model and are caught by length-check rather than BCH math — count the characters on a plate before decoding.

Cross-card recovery (asymmetric). Public material (`mk1`, `md1`) is re-derivable from a surviving `ms1` plus knowledge of the wallet template. A lost or destroyed `ms1`, however, **cannot be reconstructed** from the public cards alone — those degrade to a watch-only wallet. For 2-of-3 multisig, the threshold adds further redundancy: any one cosigner's `ms1` may be lost without losing spending capability; two cannot. The reference manual's recovery-paths chapter walks through every scenario; appendix E of the manual cites the BIP-93 numbers in full.

The toolkit verifies that the three cards belong together via a small fingerprint called the **policy ID stub**: a 4-byte hash of the wallet policy that each `mk1` and `md1` card carries at encode time, so mixing cards from different wallets is caught immediately. If the stubs disagree, `mnemonic verify-bundle` fails fast.

Onward: install the toolkit and produce your first single-sig bundle.

Install the toolkit

You need three command-line binaries: `mnemonic`, `md`, and `ms`. This Quick Start uses all three. The recommended path until the crates land on crates.io is `cargo install --git`.

Pre-requisites

A recent **Rust toolchain** (1.77 or newer). If you don't have one, the easiest install is:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Verify with `cargo --version` and `rustc --version`.

Install the three binaries

```
cargo install --locked --git https://github.com/bg002h/mnemonic-toolkit.git
↪  mnemonic-toolkit
cargo install --locked --git https://github.com/bg002h/descriptor-mnemonic.git md-cli
cargo install --locked --git https://github.com/bg002h/mnemonic-secret.git ms-cli
```

Each command compiles from source and writes the binary into `~/.cargo/bin/`. Make sure that directory is on your `PATH` (rustup adds it automatically on most platforms).

Smoke check

```
mnemonic --version
md --version
ms --version
```

You should see a version line from each. `mnemonic --version` must report `0.8.0` or later — earlier versions used a different flag set than the rest of this guide.

The reference manual covers Docker and from-source build paths; this Quick Start sticks to the cargo path.

Onward: generate the entropy you'll feed into your first bundle.

Generating entropy safely

Before you produce a bundle you need a fresh, secret seed phrase. “Fresh” means generated by something that picks bits at random in a way an attacker cannot predict; “secret” means the bits exist on a device disconnected from the network for as long as they’re visible. The imperative: **don’t make up the words yourself, and don’t generate them on a machine that has ever touched the internet during the ceremony.**

Why fresh entropy matters

A 12-word BIP-39 phrase encodes 128 bits of randomness. If you pick the words yourself — even creatively, even from a list — the entropy collapses to whatever your brain can do, which is far less than 128 bits. Wallets seeded from human-chosen phrases get drained within minutes of being funded. The phrase has to come from a real random source.

How to generate entropy in production

Pick one of:

- **A hardware wallet’s “new seed” flow.** The wallet’s secure element samples bits from an on-chip random source and displays the resulting 12 / 24 words on its screen. Write them down on scratch paper for the duration of the ceremony.
- **An air-gapped computer running an offline BIP-39 generator.** A laptop booted from a read-only USB image, with the network disabled, running a vetted generator (`bitcoinjs-lib` in a browser, the Coldcard’s dice-rolling mode, or similar).

Either way the imperative is the same: the bits are sampled on a device with no network reach, displayed only long enough to write down and engrave, and never typed into a connected computer.

The toolkit accepts any standards-conforming 12 / 15 / 18 / 21 / 24-word mnemonic in any of the ten BIP-39 wordlists.

> Every command in this Quick Start uses the canonical BIP-39 test vector `abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon`

`abandon about`. This phrase is **public** — it appears in BIP-39's test vectors, so chain watchers have indexed every wallet ever derived from it and sweep funds from such wallets within seconds. **Never engrave or fund a wallet built from this phrase.** Generate fresh entropy for any real bundle.

Using the canonical test seed for this guide

We use the public test phrase deliberately: every command in the following chapters has a deterministic, copy-pasteable output, so you can follow along by typing the commands verbatim and comparing the results to the printed transcripts. When you build a real bundle, replace the `--slot @0.phrase=…` argument with your fresh phrase and the rest of the workflow is identical.

Onward: produce your first bundle.

Producing your first bundle

This chapter produces a complete 3-card bundle for a single-sig BIP-84 mainnet wallet. End-to-end takes about three minutes.

The command

> **Reminder.** The phrase below is the public BIP-39 test vector — never engrave or fund a wallet built from it. See Generating entropy safely for the full warning.

```
mnemonic bundle \
  --network mainnet \
  --template bip84 \
  --slot @0.phrase="abandon abandon abandon abandon abandon abandon abandon abandon abandon
  ↪  abandon abandon about"
```

Three flags carry the work:

- **`--network mainnet`** — the wallet is for Bitcoin mainnet. Other values: `testnet`, `signet`, `regtest`.
- **`--template bip84`** — the wallet's spending rule. `bip84` is single-sig native SegWit (P2WPKH) at derivation `m/84'/0'/0'`. Other single-sig templates: `bip44` (legacy), `bip49` (nested-segwit), `bip86` (taproot).
- **`--slot @0.phrase=…`** — the seed-bearing input for cosigner index `@0`. Single-sig has only one cosigner, so only `@0`. The multisig walkthrough in Part III uses `@0`, `@1`, `@2`.

The `@N` notation is the toolkit's way of indexing a slot in a wallet template. For single-sig there is only `@0`. For 2-of-3 multisig there are `@0`, `@1`, `@2` — one per cosigner. Inputs attach to slots via `--slot @N.<subkey>=<value>` where `<subkey>` can be `phrase` (BIP-39 seed phrase), `xpub` (watch-only xpub), `entropy` (raw bytes), `wif` (single-key import), and a few more. The full subkey grammar lives in `mnemonic bundle --help`.

For a real bundle, replace the canonical phrase with the one you generated in Generating entropy safely.

Output

Three card strings, each shown twice — once contiguous (handy for copy-paste) and once chunked in 5-character groups (handy for engraving):

```
# ms1 (entropy, BCH-checksummed)
ms10entrsqqqqqqqqqqqqqqqqqqqqqqqqqqqqcj9sxraq34v7f

ms10e ntrsq qqqqq qqqqq qqqqq qqqqq qqqqq qqcj9 sxraq 34v7f

# mk1 (xpub + origin)

mk1qprsqhpqqsq3cqtsleeutks2qvzg3vs70mejhk622ws2kgdemj2cd8zwj2skzx2wq0qw70l4q99vdyh5x0z8v4yslsp8qp3yxg3
mk1qprsqhpp0f30mtxzd65mvwcur9usdatwuqvq6z70r9nwrgk6xn6l8gy6nwa2n977sw6zh34rma0nh

mk1qp rsqhp qqsq3 cqtsl eeutk s2qvz g3vs7 0mejh k622w s2kgd
emj2c d8zwj 2skzx 2wq0q w70l4 q99vd yh5x0 z8v4y slsp8 qp3yx
g3dpe 854wq 4
mk1qp rsqhp p0f30 mtxzd 65mvw cur9u sdatw uqvq6 z70r9 nwrgk
6xn6l 8gy6n wa2n9 77sw6 zh34r ma0nh

# md1 (wallet policy)
md1zsxdspqqqpm6jzzqqvqz6qu79mg9p2sgfff6p2eph8wftp5uf6gqnlgzqqqnymv0
md1zsxdspq259s3jnsrcrhnlagpftrf9apnc3m9fy8uqfc85cha4nqnh5k67ey2hzyc
md1zsxdspqjd65mvwcur9usdatwuqvq6z70r9nwrgk6xn6l8gy6nvqhuuyvzgaejah6

md1zs-xdspq-qqpm6-jzzqq-vqz6q-u79mg-9p2sg-fff6p-2eph8-wftp5-uf6gq-nlgzq-qqnym-v0
md1zs-xdspq-259s3-jnsrc-rhnla-gpftr-f9apn-c3m9f-y8uqf-c85ch-a4nqn-h5k67-ey2hz-yc
md1zs-xdspq-jd65m-vwcur-9usda-twuqv-q6z70-r9nwr-gk6xn-6l8gy-6nvqh-uuyvz-gaeja-h6

# === Wallet bundle: bip84, mainnet ===
# ms1: 1c017
# mk1: 1c017
# fingerprint: 73c5da0a
# origin path: 84'/0'/0'
# Template: bip84
# md1: 1c01
warning: secret material on stdout — consider redirecting (e.g., '> file.txt' or '| age -e
↪  ...')
```

Reading the output

Each card section opens with a header comment, then the contiguous form, then the chunked form. The trailing block beginning `# === Wallet bundle:` is a metadata summary, not part of the engraving.

Card	Contiguous form	What it is
ms1	`ms10entrsqq…34v7f` (1 string)	Goes on the *secret* card. Carries the BIP-39 entropy.

Card	Contiguous form	What it is
mk1	`mk1qprsqhp…854wq4` and `mk1qprsqhp…ma0nh` (2 strings)	Goes on the *key* card. The xpub + origin is too long for one BCH-checksummed group, so it splits across two strings.
md1	`md1zsxdspq…` (3 strings)	Goes on the *descriptor* card. Encodes the wallet policy and binds it to the xpub.

The metadata block at the end shows the version stamps (`1c017`), master fingerprint, origin path, template, and md1 version — useful for visually comparing two bundles, never engraved.

The trailing `warning: secret material on stdout` is real. For a real bundle, redirect to a file (`> bundle.txt`) and either delete the file after engraving or pipe through `age -e` to encrypt at rest. The secret here is your seed phrase and the ms1 card derived from it.

What about the engraving cards themselves?

The toolkit also emits an *engraving-card* layout to stderr — a visual aid showing where each chunk goes on the physical plate. The cards covered in Part II (Stamping the steel plates) walk through using the layout when transferring strings to steel. Pass `--no-engraving-card` to suppress the stderr emission if you don't want it.

Onward: verify the bundle round-trips before engraving.

Verifying the bundle

Run `mnemonic verify-bundle` *before* you engrave. It re-derives the expected card content from the seed and confirms the bundle you produced matches.

The command

Pass the same `--network`, `--template`, and `--slot @0.phrase=…` as the original `bundle` invocation, plus the cards that came back in the bundle output:

> **Reminder.** Still the public BIP-39 test phrase — see Generating entropy safely.

```
mnemonic verify-bundle \
  --network mainnet \
  --template bip84 \
  --slot @0.phrase="abandon abandon abandon abandon abandon abandon abandon abandon abandon
  ↪  abandon abandon about" \
  --ms1 ms10entrsqqqqqqqqqqqqqqqqqqqqqqqqqqqcj9sxraq34v7f \
  --mk1
  ↪  mk1qprsqhpqqsq3cqtsleeutks2qvzg3vs70mejhk622ws2kgdemj2cd8zwj2skzx2wq0qw70l4q99vdyh5x0z8v4yslsp8qp3y
  ↪  \
  --mk1 mk1qprsqhpp0f30mtxzd65mvwcur9usdatwuqvq6z70r9nwrgk6xn6l8gy6nwa2n977sw6zh34rma0nh \
  --md1 md1zsxdspqqqpm6jzzqqvqz6qu79mg9p2sgfff6p2eph8wftp5uf6gqnlgzqqqnymv0 \
  --md1 md1zsxdspq259s3jnsrcrhnlagpftrf9apnc3m9fy8uqfc85cha4nqnh5k67ey2hzyc \
  --md1 md1zsxdspqjd65mvwcur9usdatwuqvq6z70r9nwrgk6xn6l8gy6nvqhuuyvzgaejah6
```

The flag set:

- **`--ms1 <STRING>`** — the single ms1 card.
- **`--mk1 <STRING>`** — repeat once per mk1 string. BIP-84 emits two, so `--mk1` appears twice.
- **`--md1 <STRING>`** — repeat once per md1 string. BIP-84 emits three, so `--md1` appears three times.

The `--network`, `--template`, and `--slot` flags must match the original `bundle` invocation so the verifier knows what to expect.

Output

```
ms1_decode: ok decoded successfully
ms1_entropy_match: ok ms1 byte-identical
mk1_decode: ok decoded successfully
```

```
mk1_xpub_match: ok xpub matches
mk1_fingerprint_match: ok fingerprint matches
mk1_path_match: ok path matches
md1_decode: ok decoded successfully
md1_wallet_policy: ok wallet-policy mode confirmed
md1_xpub_match: ok 65-byte xpub matches expected
result: ok
```

Each line is a discrete check. `*_decode` lines confirm the BCH checksum verifies and the card parses. The `*_match` lines confirm the decoded content equals what the seed and template would produce at this network. `result: ok` on the final line is the green light.

If something is wrong — a transcription typo, a flag mismatch with the original bundle, a card from a different wallet — the failing sub-check names what disagrees and the run exits with a non-zero status.

> When a `*_decode` failure includes a *position* — for instance `mk1_decode: error at position 47` — the number is a 0-based index into the card string and pinpoints the single character whose checksum disagrees. This is the BCH code's locator output: a single-character error can be detected and located precisely. So "position 47" means look at character 47 of the failing card, re-stamp it, and re-run the check. You don't have to guess which character broke; the diagnostic names it.

Why this matters before engraving

Verification protects against three failure modes you do *not* want to discover after stamping steel:

1. **Transcription errors.** A copy-paste typo flips one character. The BCH checksum on the affected card catches it; `*_decode` fails and the diagnostic names the position.
2. **Wrong template or network.** If you bundled `bip84` mainnet but mistakenly invoke `verify-bundle` with `--template bip49`, every `*_xpub_match` fails because the verifier re-derives at the wrong path.
3. **Mixed cards from different wallets.** The cross-binding via `policy_id_stub` means an mk1 from one bundle can't be combined with an md1 from another. Mixed cards fail the cross-check.

Run `verify-bundle` once after every bundle synthesis. If `result: ok`, you are safe to stamp.

Onward: stamp the cards onto steel.

Stamping the steel plates

Engraving turns three card strings on a screen into three steel plates that survive fire, flood, and time. The discipline is short: stamp all three plates, then re-decode them as a set through `mnemonic verify-bundle`.

Ceremony at a glance

The chapters before this one cover the first three boxes (entropy → bundle → verify). This chapter covers the rest.

Stamping discipline

- **Pick the right steel.** Use a plate rated for the heat of the worst fire you're insuring against — most products advertise the rating in degrees Celsius. The toolkit's character set is deliberately limited to letters and digits that stamp cleanly, so almost any modern engraving plate works.
- **Use a magnifier.** The codec alphabet excludes visually-similar characters (no `0`/`O`, no `1`/`l`), but stamping artefacts can still mimic the wrong character at a glance. A 5x loupe pays for itself the first time it catches a misaligned strike.
- **Re-decode the set after stamping.** Don't trust your own typing or your own striking. Once all three plates are stamped, read the characters off each plate and run `mnemonic verify-bundle` with the steel-read strings (same `--ms1`/`--mk1`/`--md1` shape as chapter 24). If `result: ok`, the set is engraved correctly; if a sub-check fails (e.g., `mk1_decode: error at position 47`), the diagnostic names the card and the character position so you know which plate to re-strike.

Where each plate goes

The convention is colour-keyed. The `mnemonic-toolkit` engraving- card emission uses red for ms1, blue for mk1, green for md1:

Plate	Card	Strings to stamp	What it carries
Red	ms1	1	BIP-39 entropy
Blue	mk1	2	xpub + origin (master fingerprint, BIP path)
Green	md1	3	wallet policy + bound xpub

Use the *chunked* form (`ms10e ntrsq qqqqq qqqqq qqqqq qqqqq qqqqq qqcj9 sxraq 34v7f`) as the engraving guide — five-character groups separated by spaces align with the printed engraving cards.

Geographic separation

Once all three plates verify, store them in three independent locations. A common arrangement:

- **Red (ms1)** — primary safe, on-site, fireproof.
- **Blue (mk1)** — bank safe-deposit box.
- **Green (md1)** — trusted family member, attorney, or off-site vault.

The cross-binding via `policy_id_stub` means an attacker who finds only one plate cannot derive the wallet alone — the mk1 and md1 together are watch-only, and the ms1 alone leaves recovery software to *guess* the spending rule. Distribute the three plates across people, jurisdictions, and threat models.

For taproot variants, `--privacy-preserving`, and the long version of this ceremony, see the reference manual's **Single-sig steel-engraved backup** workflow chapter.

Onward: practice recovery from the engraved plates.

Recovering from the plates

Recovery is the inverse of Producing your first bundle: read the cards off the engraved plates and reconstruct what you need to spend or watch the wallet. Three commands, one per card.

What you have, what you want

In hand: three engraved plates with the card strings from Producing your first bundle. Wanted:

- the seed phrase (to sign spends),
- the xpub plus origin (to construct addresses),
- the wallet policy (to know *what kind* of wallet to recover into).

Step 1 — recover the phrase from ms1

> **Reminder.** This Quick Start uses the public BIP-39 test phrase throughout — including in the output below. See Generating entropy safely for why you must never use it for a real wallet.

```
mnemonic convert \
  --from ms1=ms10entrsqqqqqqqqqqqqqqqqqqqqqqqqqqqcj9sxraq34v7f \
  --to phrase
```

Output:

```
phrase: abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon
↪  abandon about
warning: secret material on stdout — consider redirecting (e.g., '> file.txt' or '| age -e
↪  ...')
```

The phrase imports directly into any BIP-39 wallet. The `warning` on stderr is a reminder that the phrase is on your terminal and should not stay there — redirect to a file or pipe through `age -e` if you need to persist it.

Step 2 — recover xpub, fingerprint, and path from mk1

The two mk1 strings are passed as a single space-separated value:

```
mnemonic convert \
  --from mk1=
  ↪  "mk1qprsqhpqqsq3cqtsleeutks2qvzg3vs70mejhk622ws2kgdemj2cd8zwj2skzx2wq0qw70l4q99vdyh5x0z8v4yslsp8qp3y
  ↪  mk1qprsqhpp0f30mtxzd65mvwcur9usdatwuqvq6z70r9nwrgk6xn6l8gy6nwa2n977sw6zh34rma0nh" \
  --to xpub --to fingerprint --to path
```

Output:

```
xpub:
 ↪  xpub6CatWdiZiodmUeTDp8LT5or8nmbKNcuyvz7WyksVFkKB4RHwCD3XyuvPEbvqAQY3rAPshWcMLoP2fMFMKHPJ4ZeZXYVUhLv1VM
fingerprint: 73c5da0a
path: 84'/0'/0'
```

The toolkit re-assembles the two strings, verifies the BCH checksum on each, and emits the three derived values. The `--to` flag repeats once per output you want — drop `--to fingerprint` and `--to path` if you only need the xpub.

Step 3 — recover the wallet policy from md1

The three md1 strings are passed *positionally* to `md decode`:

```
md decode \
  md1zsxdspqqqpm6jzzqqvqz6qu79mg9p2sgfff6p2eph8wftp5uf6gqnlgzqqqnymv0 \
  md1zsxdspq259s3jnsrcrhnlagpftrf9apnc3m9fy8uqfc85cha4nqnh5k67ey2hzyc \
  md1zsxdspqjd65mvwcur9usdatwuqvq6z70r9nwrgk6xn6l8gy6nvqhuuyvzgaejah6
```

Output:

```
wpkh(@0/<0;1>/*)
```

`wpkh(@0/<0;1>/*)` is the BIP-388 wallet policy template for a BIP-84 single-sig wallet: one key (`@0`), native segwit (`wpkh`), with `<0;1>` covering the external-and-change descriptor pair.

Putting it together

You now have everything a watch-only wallet needs (xpub, path, fingerprint, template) and everything a signer needs (the phrase). Re-running `mnemonic verify-bundle` with the recovered values is a useful final sanity check — it exercises the same `policy_id_stub` cross-binding as the original verification and confirms the three plates still belong together.

If verification passes, import the phrase into your signing wallet and the xpub-plus-policy into your watch-only wallet of choice. Part IV — watch-only covers the import shapes for Bitcoin Core, Sparrow, and Specter.

For multisig (a 2-of-3 wallet built from three independent cosigners), continue with Part III — multisig.

Why multisig

Single-sig is one seed, one signer, one point of failure. Multisig splits signing authority across several independent cosigners, so losing — or compromising — any one of them does not lose the wallet. This chapter motivates the 2-of-3 layout the rest of Part III walks through.

When single-sig isn't enough

A BIP-84 single-sig wallet is recovered by anyone who reads the seed phrase. That is its whole security model: the phrase is the wallet. The model breaks under any of these:

- **Compromise.** Someone photographs the steel plate, sees the phrase on a connected screen, or coerces it out of the holder.
- **Single-point loss.** Fire takes the only plate; the holder forgets where they hid it; the holder dies without sharing it.
- **Insider trust.** A custodian, family member, or business partner who handles the phrase can drain the wallet unilaterally.

There is no defense against any of these in single-sig. The Stamping the steel plates ceremony helps with *durability* (geographic separation of `ms1`/`mk1`/`md1`) but not with the underlying "one seed = one signer" structure.

What 2-of-3 means

A *K-of-N* multisig wallet has `N` cosigners, any `K` of whom can sign together to spend. The remaining `N − K` cosigners contribute nothing to a given spend. For 2-of-3:

- Three cosigners, each holding their own independently-generated seed.
- Any two of them, cooperating, can spend.
- A single cosigner, alone, cannot spend.

The wallet's address is derived from *all three* cosigner xpubs plus the spending rule, so the receive side is unchanged from the user's perspective — they hand out one address as usual. The signing side is what differs: a spend transaction collects two of the three cosigners' signatures, and the network accepts it.

Air-gapped vs. coordinated

Two operational modes for producing the bundle:

- **Coordinated.** One laptop sees all three phrases briefly during the bundle pass. Faster; trusts the coordinator's machine for the duration of the synthesis.
- **Air-gapped.** Each cosigner runs a `convert` step on their own machine to derive *only* their xpub; the coordinator builds a watch-only bundle from the three xpubs (no secrets ever leave any cosigner's machine); each cosigner separately derives their own ms1 locally.

Part III walks through the **coordinated** flow because it is the shorter, copy-pasteable shape suited to a Quick Start. The full air-gapped variant is in Part IV — watch-only multisig and the reference manual's multisig workflow chapter.

Why 2-of-3 specifically

The "2-of-3" choice is the canonical small-team layout for two reasons:

- **1-of-N defeats the purpose.** Any single cosigner spending alone is identical to single-sig with extra steps. The whole point of multisig is that no individual key is sufficient.
- **K = N loses the recovery property.** A 3-of-3 wallet is bricked the moment any one cosigner is unavailable. With 2-of-3, losing one cosigner is survivable: the other two still meet threshold.

Larger layouts (3-of-5, 4-of-7) extend the same logic; the toolkit caps at `1 ≤ K ≤ N ≤ 16`. For most personal and small-business setups, 2-of-3 is the sweet spot: one cosigner per geography or per trust domain, with one redundant slot.

Onward: produce the seven-card 2-of-3 bundle.

Producing a 2-of-3 bundle

This chapter produces the full seven-card bundle for a 2-of-3 segwit multisig wallet on mainnet. Three cosigners contribute one phrase each; the toolkit emits three ms1 cards (one per cosigner), three mk1 cards (one per cosigner), and one shared md1 card.

What you build

Three phrases in, **seven cards out**: 3 × ms1 + 3 × mk1 + 1 × md1. Each cosigner's personal engraving set is *their own* ms1 plus all three mk1s plus the one shared md1 — five plates per cosigner, seven plates total across the wallet.

The command

> The phrases below are **public BIP-39 test vectors**. They appear verbatim in the BIP-39 specification, so chain watchers have indexed every wallet ever derived from them and sweep funds within seconds of receiving any. **Never engrave or fund a wallet built from these phrases.** For a real bundle, generate three fresh seeds — one per cosigner, on three separate air-gapped devices — using the same discipline as Generating entropy safely.

```
mnemonic bundle \
  --network mainnet \
  --template wsh-sortedmulti \
  --threshold 2 \
  --slot @0.phrase="abandon abandon abandon abandon abandon abandon abandon abandon abandon
  ↪  abandon abandon about" \
  --slot @1.phrase="legal winner thank year wave sausage worth useful legal winner thank
  ↪  yellow" \
  --slot @2.phrase="letter advice cage absurd amount doctor acoustic avoid letter advice
  ↪  cage above" \
  --self-check
```

Four flags carry the multisig-specific work:

- **`--template wsh-sortedmulti`** — the wallet's spending rule. `wsh-sortedmulti` is BIP-67-sorted segwit multisig; the emitted descriptor is `wsh(sortedmulti(2,@0,@1,@2))`. The "sorted" part matters during recovery: BIP-67 fixes a canonical cosigner order so losing track of "who was @0?" doesn't brick the wallet. Other multisig

templates exist (`wsh-multi` unsorted, `sh-wsh-…` nested-segwit, `tr-…` taproot); `wsh-sortedmulti` is the conventional choice for new wallets.
- **`--threshold 2`** — the K of K-of-N. Required for any multisig template; ignored for single-sig. The toolkit enforces `1 ≤ K ≤ N ≤ 16`.
- **`--slot @N.phrase=…`** repeated three times — one slot per cosigner, indexed `@0` through `@2`. The slot index becomes the cosigner index in the BIP-388 wallet policy. Single-sig used only `@0`; multisig adds `@1`, `@2`, etc.
- **`--self-check`** — re-parses the emitted bundle and verifies it round-trips. A built-in sanity check that catches encode/decode drift before the cards reach steel.

Output

The toolkit emits seven cards on stdout, grouped by card type, plus a metadata summary at the end. The structure mirrors the single-sig bundle from chapter 23, with an added `# === Cosigner N ===` separator preceding each cosigner's ms1/mk1 strings. (Exact strings are deterministic for the test phrases above and are spelled out in the manual's full multisig chapter; see the cross-reference at the end of this chapter.)

Card	Count	What it carries
ms1	3 (one per cosigner)	BIP-39 entropy for that cosigner's seed
mk1	3 (one per cosigner)	xpub + origin for that cosigner
md1	1 (shared)	wallet policy `wsh(sortedmulti(2,…))` + bound xpubs

The trailing metadata block lists each cosigner's master fingerprint and origin path, plus the wallet's `policy_id_stub` — the cross-binding hash that every mk1 and the md1 carry, so mixing cards from different bundles is caught at verification time.

The same `warning: secret material on stdout` reminder from the single-sig chapter applies, three times over: each cosigner's ms1 contains their seed in encoded form. For a real bundle, redirect to a file (`> bundle.txt`) and either delete the file after engraving or pipe through `age -e` to encrypt at rest.

What's next

The companion `mnemonic verify-bundle` step (mirroring chapter 24) takes the same four re-derivation flags plus one repetition per *string* in the output: three `--ms1` (3 cosigner secrets, 1 string each), six `--mk1` (3 mk1 cards, 2 strings each), and four `--md1` (1 md1 card, ~4 strings for the longer wsh-sortedmulti descriptor). The flag-count tracks the chunked-string emission, not the seven-plate card count.

The full air-gapped variant — where each cosigner produces their own ms1 + xpub locally and the coordinator only ever sees public xpubs — is in Part IV — watch-only

multisig.

Onward: stamp the per-cosigner plate sets and learn the recovery table.

Stamping and recovering a 2-of-3 wallet

Each cosigner walks away from the bundle with **five plates**: their own ms1 plus the three shared mk1s plus the one shared md1. This chapter covers the per-cosigner plate set and the recovery quick-table — what is still spendable, what is watch-only, and what is bricked across the common damage scenarios.

Per-cosigner plate set

> **Reminder.** Examples below still reference the public BIP-39 test phrases used in chapter 32's bundle — see the DANGER box there for why they must never back a real wallet.

After stamping, every cosigner holds the same shape of set. Cosigner N's plates:

Plate	Held by	Engraved with
Red (ms1)	only cosigner N	their *own* ms1 (1 string)
Blue × 3 (mk1)	every cosigner	the three mk1 cards (2 strings each)
Green (md1)	every cosigner	the shared md1 (3-4 strings)

The mk1 plates are public xpub material — distributing copies across all three cosigners is harmless and useful (any cosigner can reconstruct the full wallet's watch-only side from their own copy). The md1 plate is also public; it carries no secrets, only the wallet policy. The single plate that must be guarded is the cosigner's own red ms1.

The stamping discipline from chapter 25 applies per plate: stamp, re-decode through `mnemonic verify-bundle` with the steel-read strings, and re-strike on any sub-check failure *before* moving to the next plate. The BCH error-position diagnostic names the failing card and the character offset.

For geographic separation: each cosigner stores their own red plate in their primary safe, and the blue/green plates in a secondary location (bank box, attorney, off-site vault). With three cosigners times five plates, the wallet is engraved across at least six independent locations.

Recovery quick-table

The single rule: **each cosigner's seed reconstructs that cosigner's own ms1 and mk1**, and the md1 (the wallet policy) is derivable from any one of the three xpubs plus the bundle's template. So recovery scenarios fan out from "how many seeds remain accessible, and is the threshold still met?".

Lost / damaged	Wallet status	Recovery path
One cosigner's red ms1	spendable (other two cosigners meet threshold)	The lost cosigner re-derives their ms1 from their seed via `mnemonic convert --from phrase=… --to ms1` and re-stamps.
One blue mk1 plate	spendable	The cosigner whose mk1 was lost re-derives their own from their seed via `mnemonic convert --from phrase=… --to mk1`; the mk1 carries no secret, but each is bound to one specific cosigner's xpub.
The green md1 plate	spendable	Re-derive from the three xpubs via `mnemonic export-wallet --template wsh-sortedmulti --threshold 2 --slot @N.xpub=…` (one slot per cosigner; no seeds needed).
Two cosigners' ms1s + the md1 still readable	spendable	The threshold is 2; any two surviving seeds spend the wallet.
Only one cosigner's ms1 + md1 readable	watch-only only	One seed cannot meet the 2-of-3 threshold; the wallet receives but does not spend. Move funds out via the surviving cosigners' coordination if any of their seeds can still be restated.
All three cosigners lose their ms1s and forget their seeds	bricked	No threshold-meeting subset of seeds survives. Funds are unrecoverable.

The "watch-only only" row is the row to study most carefully: one seed is sufficient to reconstruct the full *public* side of the wallet (addresses, balances, transaction history)

but not to sign. Recovering from that state requires at least one more cosigner to restate their seed, after which the surviving two meet threshold and can spend.

For the long form of these scenarios — including what to do after recovery, key rotation, and partial card damage within a single plate — see the reference manual's **Recovery paths by damaged-card scenario** workflow chapter.

Onward

You have produced a 2-of-3 multisig bundle, stamped its seven plates across three cosigners, and learned the recovery quick-table. Part IV — watch-only covers importing the public side of the wallet (xpubs + policy) into Bitcoin Core, Sparrow, and Specter so addresses can be generated and balances watched without ever exposing a seed.

Watch-only single-sig

A *watch-only* wallet sees the chain — incoming payments, balance, address derivation — without holding the seed. The m-format constellation supports this via a **2-card bundle**: mk1 + md1, no ms1. Without the secret card the wallet cannot sign, but Bitcoin Core, Sparrow, or Specter can derive addresses and watch the chain.

Use this when you want to monitor a wallet from an internet-connected host while the seed stays on an air-gapped device — for incoming-payment tracking, balance reporting, or coordinator workflows that present PSBTs to remote signers.

Two-step shape

The seed never crosses to the bundle-building host. Step 1 derives the xpub on the seed-holding machine; step 2 builds the bundle from the xpub alone on a separate (potentially internet-connected) host.

> **Reminder.** The phrase below is the public BIP-39 test vector used throughout this Quick Start — never use it for a real wallet. See Generating entropy safely.

Step 1 — derive the xpub (on the air-gapped seed-holder)

```
mnemonic convert \
  --from phrase="abandon abandon abandon abandon abandon abandon abandon abandon abandon
  ↪  abandon abandon about" \
  --to xpub \
  --template bip84 \
  --network mainnet
```

Output:

```
xpub:
↪  xpub6CatWdiZiodmUeTDp8LT5or8nmbKNcuyvz7WyksVFkKB4RHwCD3XyuvPEbvqAQY3rAPshWcMLoP2fMFMKHPJ4ZeZXYVUhLv1VM
```

Hand-carry that xpub string to the bundle-building host (it is public information — paper, USB stick, QR code, or just retyping all work).

Step 2 — build the 2-card bundle (on any host)

```
mnemonic bundle \
  --network mainnet \
  --template bip84 \
```

```
--slot
↪  @0.xpub=xpub6CatWdiZiodmUeTDp8LT5or8nmbKNcuyvz7WyksVFkKB4RHwCD3XyuvPEbvqAQY3rAPshWcMLoP2fMFMKHPJ4ZeZ
```

Because the slot input is `xpub` rather than `phrase`, no entropy exists for the toolkit to encode — the output is **2 cards**: one mk1 (xpub + origin) and one md1 (wallet policy). No ms1, no secret material on stdout, no `warning: secret material` line.

The same `--template`, `--network`, and `--account` choices from chapter 23 apply — the only difference is the slot input shape (`@0.xpub=…` instead of `@0.phrase=…`).

What you get

Card	Strings	Holds
mk1	2	xpub + origin (master fingerprint + path)
md1	3	wallet policy `wpkh(@0/<0;1>/*)` bound to the xpub

Stamp the two plates using the same discipline from chapter 25. Recovery from mk1 + md1 alone (`mnemonic convert --from mk1=… --to xpub --to fingerprint --to path` plus `md decode <md1 strings>`) yields everything a watch-only client needs.

Importing into a wallet

Once you have the watch-only bundle (or recovered xpub + policy from the plates), turn it into the artifact your monitoring software wants — Bitcoin Core `importdescriptors` JSON, BIP-388 wallet policy, or a wallet-specific shape — using `mnemonic export-wallet`. The reference manual's Exporting to Bitcoin Core / BIP-388 / Sparrow / Specter chapter walks through the four output formats and their import flows.

Onward: the air-gapped multisig variant, where each cosigner derives their own xpub locally and only public material crosses to the coordinator.

Watch-only multisig (air-gapped)

Chapter 32 walked the *coordinated* multisig flow — one laptop briefly sees all three phrases. This chapter walks the **air-gapped** variant: each cosigner derives their own xpub on their own machine, only xpubs cross to the coordinator, and the coordinator builds a watch-only bundle. No seed ever leaves its origin machine.

Air-gapped synthesis

The boundary that matters: only xpubs cross from the cosigner machines to the coordinator. The coordinator never holds enough material to spend.

Step 1 — each cosigner derives their xpub locally

On each cosigner's air-gapped machine, derive the xpub from that cosigner's own seed:

```
mnemonic convert \
  --from phrase="<this cosigner's phrase>" \
  --to xpub --to fingerprint \
  --template wsh-sortedmulti \
  --network mainnet
```

The cosigner writes down the xpub plus master fingerprint and hand-carries them (paper, QR, USB) to the coordinator. The phrase stays on the air-gapped machine.

For BIP-87 paths (the default `--multisig-path-family` for the toolkit's multisig templates), the path is `m/87'/0'/0'`; for Coldcard / SeedSigner / older-Sparrow compatibility add `--multisig-path-family bip48` and the path becomes `m/48'/0'/0'/2'`.

Step 2 — coordinator builds the watch-only bundle

Once the coordinator has all three xpubs, build the bundle from xpubs only:

```
mnemonic bundle \
  --network mainnet \
  --template wsh-sortedmulti \
  --threshold 2 \
  --slot @0.xpub=<xpub-cosigner-0> \
  --slot @1.xpub=<xpub-cosigner-1> \
  --slot @2.xpub=<xpub-cosigner-2>
```

The output is **4 cards**: 3 mk1 (one per cosigner's xpub + origin) and 1 md1 (the shared wallet policy `wsh(sortedmulti(2,@0,@1,@2))`). No ms1 cards — no secret material was passed.

For coordinator-side bundles that fan out to many cosigners and should not reveal each cosigner's master fingerprint, add `--privacy-preserving`; mk1s emit a zero-fingerprint placeholder that recovery software re-derives at import time.

Step 3 — each cosigner separately derives their own ms1

The watch-only bundle is the public-side artifact. To be able to sign, each cosigner derives their *own* ms1 on their own air-gapped machine:

```
# On cosigner N's air-gapped machine
mnemonic convert \
  --from phrase="<cosigner N's phrase>" \
  --to ms1
```

Each cosigner stamps their own ms1 plate alongside copies of the 3 mk1 plates and the shared md1 plate. The result is identical to the chapter 33 per-cosigner plate set — same five plates each, same recovery quick-table — but produced without ever exposing more than one seed to any single machine.

What's next

The coordinator now holds a 4-card watch-only bundle (3 mk1 + 1 md1) suitable for monitoring the wallet. To turn it into a Bitcoin Core `importdescriptors` JSON or a BIP-388 wallet policy, pass the same slot inputs to `mnemonic export-wallet` — same flag shape, different output format. See the manual's Multisig 2-of-3 walkthrough for the canonical full air-gapped procedure (including signing ceremonies and PSBT-routing across the coordinator boundary).

Onward: pointers into the reference manual for going deeper.

Where to go from here

This Quick Start covered single-sig, multisig, and watch-only by worked example. The reference manual goes deeper on every topic. Topic-keyed pointers below.

Going deeper on workflows

The manual's *Workflows* part walks each end-to-end ceremony in full, including operational variants this Quick Start skipped:

- Single-sig steel-engraving ceremony — production-quality stamping, geographic separation, re-stamp-on-failure discipline.
- Multisig 2-of-3 walkthrough — canonical air-gapped multisig including PSBT routing and signing ceremony.
- Taproot multisig — `tr-multi-a` / `tr-sortedmulti-a` and the `--taproot-internal-key` decision.
- Watch-only xpub bundle — full watch-only chapter, including the privacy-preserving variant.
- Recovery paths by damaged-card scenario — exhaustive damage matrix, including partial-card-damage handling.
- Migration from BIP-39 / Shamir / SeedQR — moving an existing wallet onto m-format steel.
- Exporting to Bitcoin Core / BIP-388 / Sparrow / Specter — the four wallet-export shapes and their import flows.
- BIP-85 child-seed derivation — generating child seeds (BIP-39, WIF, xprv, DICE) from a master.

CLI reference

For the per-flag, per-subcommand canonical reference:

- mnemonic CLI reference — every `mnemonic` subcommand, every flag, every output mode.
- md CLI reference — `md` (descriptor card) subcommands.
- ms CLI reference — `ms` (secret card) subcommands.
- mk-codec Rust API — for embedding the mk1 codec into another tool.

Comparing m-format with other backup standards

When choosing between m-format and an existing BIP / SLIP / vendor format:

- Picking a backup format — the decision tree.
- mnemonic-toolkit vs. ms-cli — when to use which.
- mnemonic-toolkit vs. md-cli — when to use which.
- m-format vs. SeedQR / Shamir / SeedSigner — head-to-head capability matrix.
- Single-sig vs. multisig — operational tradeoffs.
- BIP-39 vs. BIP-38 passphrase models — when each adds security and when it adds risk.
- Bitcoin Core descriptors vs. BIP-388 wallet policies — interchange-format choice.

BIP primers

If a referenced BIP is unfamiliar, the manual's primers cover the four most-cited:

- BIP-39 primer — entropy → phrase → seed.
- BIP-32 primer — hierarchical deterministic derivation, xpubs and xprvs.
- Descriptors primer — Bitcoin Core descriptor language.
- BCH / codex32 primer — the BCH error-correction code under m-format and codex32.

Troubleshooting full matrix

The next chapter covers the five most common newcomer issues. For the long form — bundle synthesis, verify-bundle, convert / recovery, engraving, wallet-export, and BIP-85 failure modes — Appendix G — Troubleshooting matrix is the full reference.

Onward: the five most common newcomer issues, each with a fix.

Troubleshooting

Five most common newcomer issues. For the full failure-mode matrix, see the manual's Appendix G — Troubleshooting matrix.

1. "I forgot `--threshold`"

```
error: a value is required for '--threshold <THRESHOLD>'
```

Multisig templates (`wsh-multi`, `wsh-sortedmulti`, `sh-wsh-multi`, `sh-wsh-sortedmulti`, `tr-multi-a`, `tr-sortedmulti-a`) require an explicit threshold `K`. Single-sig templates (`bip44`, `bip49`, `bip84`, `bip86`) do not.

Fix: add `--threshold K` (with `1 ≤ K ≤ N ≤ 16`). For the 2-of-3 walkthrough in chapter 32, `K = 2`.

2. "verify-bundle says ms1_decode error at position N"

```
ms1_decode: error at position 27
```

A character at position 27 of the ms1 string is wrong — typically a mis-stamped or mis-typed character on the engraved plate. The BCH checksum localised the failure to that one position; the m-format alphabet excludes visually-similar characters (`0` vs `o`, `1` vs `l`/`i`), so most errors trace to a stamping artefact rather than genuine confusion.

Fix: look at character N on the original digital bundle (or the re-derived ms1 from `mnemonic convert --from phrase=… --to ms1`), compare to the steel plate, and re-stamp that one character. Re-run `verify-bundle` after the correction.

The same pattern applies to `mk1_decode` and `md1_decode` failures — read the position N off the diagnostic, find that character on the plate, re-stamp.

3. "Bitcoin Core won't import"

```
error: Descriptor not in canonical form
```

Bitcoin Core 25+ accepts the toolkit's default `--format bitcoin-core` output directly; older Bitcoin Core versions need the v24 shape, and BIP-388 wallet-policy importers (Sparrow, Specter, Coldcard, Ledger) need a different format entirely.

Fix: re-run `mnemonic export-wallet` with the format matched to the receiving software:

- Bitcoin Core 25+ → `--format bitcoin-core` (default).
- Bitcoin Core 24 → `--format bitcoin-core --bitcoin-core-version 24`.
- Sparrow / Specter / Coldcard / Ledger → `--format bip388`.

Note: `--format sparrow` and `--format specter` are reserved values that currently return a deferral stub — use `--format bip388` for those wallets and import via their BIP-388-aware path. The manual's Exporting to Bitcoin Core / BIP-388 / Sparrow / Specter chapter covers the import-shape matrix.

4. "Wrong xpub for my wallet"

`mnemonic bundle` produced a perfectly-valid bundle, but the addresses don't match the wallet you intended to back up. The toolkit derived a different xpub than your existing wallet uses.

Cause: mismatched `--template`, `--account`, or `--network`. The xpub depends on all three; changing any one produces a different xpub. Common pitfalls:

- BIP-44 vs. BIP-49 vs. BIP-84 vs. BIP-86 — each emits a different xpub at a different derivation path.
- `--account 0` (default) vs. `--account 1` — non-zero accounts derive at `m/<purpose>'/<coin>'/N'` for the selected `N`.
- `mainnet` vs. `testnet` — different network identifiers, different xpub prefixes.

Fix: before stamping, run `mnemonic convert --from phrase=… --to xpub` with the same `--template`, `--account`, and `--network` as your bundle invocation, and compare to your wallet's xpub. If they match, the bundle is correct; if not, adjust the flags to match the wallet you're backing up.

5. "I'm on Windows and the symlinks broke"

The Quick Start and the reference manual share files via git symlinks (e.g., the worked-example transcripts). Windows clones default to leaving symlinks as plain text files containing the path target, which breaks builds and reads.

Fix: enable symlink support in git:

```
git config --global core.symlinks true
git checkout -- .  # re-materialise as symlinks
```

Symlink creation on Windows requires either Developer Mode enabled or running git as administrator. The Microsoft documentation on "Enable Developer Mode" walks through the setting; `core.symlinks=true` on its own is necessary but not sufficient.

When in doubt

1. **Re-read the chapter.** Most failures trace to a small flag-set discrepancy between the example and the actual command.
2. **Run `--help` directly.** The cli-help snapshots in the manual match toolkit v0.8.0; later versions may have added flags.
3. **Run `verify-bundle` on the engraved cards.** It is the single most useful diagnostic — the BCH error-position diagnostic names the failing card and the character offset.

For the full failure-mode matrix — bundle synthesis, verify-bundle, convert / recovery, engraving, wallet-export, and BIP-85 — see Appendix G — Troubleshooting matrix.

Build banner

Built from commit `5f01a35` on `2026-05-08T19:14:54Z` for toolkit `manual-v0.1.5`.

This page is included in the PDF render only.